CIS520
Project 4
Group 8

Zeke Flippo
Vixi Spellman
TJ Tiede

Problem Statement

Given an ASCII encoded file, our goal is write a program that for each line of the input (in order) prints a corresponding line to the output “{line_number}: {max_char}” where max_char is the greatest byte on the line (not including the newline character).

High Level Architecture

Each of our three programs share the same high level architecture. Their differences are discussed in the following section. The fundamental idea is that we minimize the amount of synchronization needed by devising a stratgy that allows each thread to operate as independently as possible.

The main thread is only responsible for:
1. Reading fixed sized batches from the input file. We read the file in batches so that we can still make progress with arbitrarily large inputs while requiring only a fixed amount of memory.
2. Making the current batch available to the worker threads
3. Receiving the worker's results and printing them in order while keeping track of how many lines have been processed
4. Maintenance of invariant 1. (see below).

We partition the batch into sections of size

$$\texttt{bytes\_per\_thread} = \left\lceil \frac{\texttt{batch\_size}}{\texttt{num\_workers}} \right\rceil \quad \text{where } \texttt{num\_workers} = \texttt{num\_threads} - 1.$$

Each worker is assigned a `worker_index` and is responsible for finding `max_char` for each of the lines that *begin* in the section starting at

$$\texttt{batch}[(\texttt{worker\_index} \cdot \texttt{bytes\_per\_thread})]$$
$$\text{and is up to and including}$$
$$\texttt{batch}[\min(\texttt{batch\_size}, \quad (\texttt{worker\_index} + 1) \cdot \texttt{bytes\_per\_thread})]$$

Each worker returns an array containing (in order) the value of `max_char` for each of the lines it is responsible for. The main thread can iterate over each thread and each `max_char` to retrieve the values in order.

For this to work we need to maintain a couple of invariants:

1. The start of the batch is always the start of a string. Besides the 0^{th} worker, all workers start at the beginning of their section but only compute `max_char` for lines after the first newline in their section (where the thread before them is responsible for that "leakage"). We require 1. so that the 0^{th} thread always has a place to start. Note that the last thread has to stop early if the last string continues into the next batch. In this case, to maintain 1. the main thread has to move those residual characters to the front of the buffer before filling it back up to make the next batch.
2. Each section needs to contain at least one newline. We rely on a newline in the threads partition to determine where its responsibility starts and another newline in the next partition to determine where its responsibility ends. In order to guarantee 2. we must have that

   $$\texttt{bytes\_per\_thread} \leq \texttt{batch\_size}$$

   which is true if and only if

   $$\texttt{max\_line\_length} \cdot \texttt{num\_workers} \leq \texttt{batch\_size}$$

   which we therefore have to maintain.

Versions Contrasted

pthreads

Under the pthreads paradigm the main() thread spawns each of worker threads which execute thread_routine(). Main passes each thread a pointer to the batch buffer and each thread returns a pointer to an array of their `max_char`'s.

MPI

Under the MPI paradigm we can not pass message by sharing memory so we must share memory by passing messages. Which is a fancy way to say that we have to copy all of the data between threads. In this way, the biggest difference in performance should be that the MPI code has to copy the batch buffer to each thread and the array of `max_char`'s from each thread.

OpenMP

OpenMP is nearly identical to pthreads except with a different a syntax that is generally less explicit.

Testing Methodology

We want to test the performance and resource usage of our code on the "mole" class of machines with different combinations of the following:

Nodes: 1, 2, 4, 8

Cores: 1, 2, 4, 8, 16, 32

Memory per core: 64MB, 128MB, 512MB, 1GB, 1.5GB, 3GB

When using `sbatch` to schedule jobs, we control the parameters

`time: uint`,

`mem-per-cpu: uint` $\times \{\mathrm{MB}, \mathrm{GB}\}$,

`cpus-per-task: uint`,

`ntasks: uint`,

`nodes: uint`,

where the total number of cores is equal to

$$\texttt{cores} = \texttt{cpus-per-task} \cdot \texttt{ntasks}.$$

Therefore for a given number of `cores` we can determine the appropriate `cpus-per-task` we can divide both sides by `ntasks` yeilding

$$\texttt{cpus-per-task} = \frac{\texttt{cores}}{\texttt{ntasks}}$$

where (since `cpus-per-task` is an integer) we must have that `cores` is a multiple of `ntasks`. Also consider that each node only has so many cores.

pthreads and OpenMP are single machine only. Under these paradigms we always have that

$$\texttt{ntasks} = 1 \quad \text{and} \quad \texttt{cpus-per-task} = \texttt{num\_threads}.$$

Reciprocally, under **MPI** we always have that

$$\texttt{ntasks} = \texttt{num\_threads} = N \quad \text{and} \quad \texttt{cpus-per-task} = 1.$$

We want to show how performance differs across multiple machines vs a single machine and using different memory "**budgets**" where

$$\texttt{budget} = \texttt{cores} \cdot \texttt{mem-per-cpu}.$$

Testing Methodology Cont.

We tested each paradigm with test cases tailored to their execution model.

pthreads and OpenMP

These pthreads and OpenMP are limited to a single-machine execution with shared memory. We test scaling on a single node with varying thread counts and memory allocations:

Configuration	Budget A: 2 GB	Budget B: 4 GB	Budget C: 8 GB
1 thread	1 node, 1 thread, 2 GB	1 node, 1 thread, 4 GB	1 node, 1 thread, 8 GB
4 threads	1 node, 4 threads, 512 MB	1 node, 4 threads, 1 GB	1 node, 4 threads, 2 GB
8 threads	1 node, 8 threads, 256 MB	1 node, 8 threads, 512 MB	1 node, 8 threads, 1 GB
16 threads	1 node, 16 threads, 128 MB	1 node, 16 threads, 256 MB	1 node, 16 threads, 512 MB

MPI

MPI can distribute work across multiple processes and nodes. We test both single-node and multi-node scenarios to isolate communication overhead:

Configuration	Budget A: 2 GB	Budget B: 4 GB	Budget C: 8 GB
1 node, 4 ranks	1 node, 4 ranks, 512 MB/rank	1 node, 4 ranks, 1 GB/rank	1 node, 4 ranks, 2 GB/rank
2 nodes, 2 ranks each	2 nodes, 2 ranks each, 512 MB/rank	2 nodes, 2 ranks each, 1 GB/rank	2 nodes, 2 ranks each, 2 GB/rank
4 nodes, 1 rank each	4 nodes, 1 rank each, 512 MB/rank	4 nodes, 1 rank each, 1 GB/rank	4 nodes, 1 rank each, 2 GB/rank
8 nodes, 1 rank each	8 nodes, 1 rank each, 256 MB/rank	8 nodes, 1 rank each, 512 MB/rank	8 nodes, 1 rank each, 1 GB/rank

Where **Budget** represents the total memory available: $\texttt{budget} = \texttt{num\_processes} \cdot \texttt{mem-per-process}$. This allows us to observe how each paradigm scales with increasing memory resources while controlling for communication overhead (MPI single-node baseline vs. multi-node scenarios) and thread scaling efficiency (pthreads/OpenMP).

Performance Analysis

Program Version	Number of Nodes	Number of Cores	Time (s)	Memory Used (MB)
pthread	1	2	6	1.92
pthread	1	4	7.3	1.76
pthread	2	2	8.2	1.92
pthread	2	4	6.4	3.784
pthread	2	8	6.1	1.76
pthread	4	4	7	1.76
pthread	4	8	5.5	1.92
MPI	1	2	8.7	19.84
MPI	1	4	10.68	20.16
MPI	2	2	71.02	19.52
MPI	2	4	131.8	19.84
MPI	2	8	238.47	20.78
MPI	4	4	316.82	19.52
MPI	4	8	269.29	21.272
OpenMP	1	2	2.27	1.6
OpenMP	1	4	1.33	1.92
OpenMP	2	2	2.2	1.6
OpenMP	2	4	1.55	1.92
OpenMP	2	8	0.93	1.92
OpenMP	4	4	1.47	1.92
OpenMP	4	8	1.16	1.92

Raw data used to compare paradigms

Performance Analysis Cont.

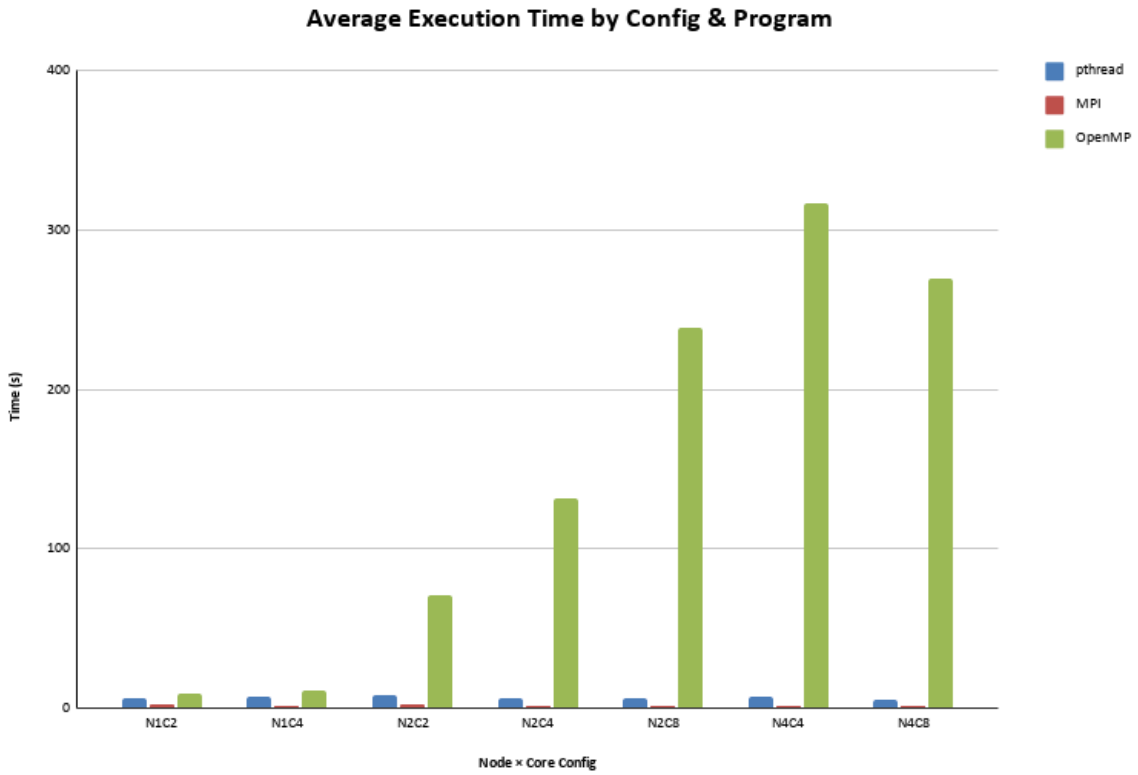
Program Type	Number of Nodes	Number of Cores	Avg Time (s)	Avg Memory (MB)
pthread	1	2	6	1.92
pthread	1	4	7.3	1.76
pthread	2	2	8.2	1.92
pthread	2	4	6.4	3.784
pthread	2	8	6.1	1.76
pthread	4	4	7	1.76
pthread	4	8	5.5	1.92
MPI	1	2	2.27	1.6
MPI	1	4	1.33	1.92
MPI	2	2	2.2	1.6
MPI	2	4	1.55	1.92
MPI	2	8	0.93	1.92
MPI	4	4	1.47	1.92
MPI	4	8	1.16	1.92
OpenMP	1	2	8.7	19.84
OpenMP	1	4	10.68	20.16
OpenMP	2	2	71.02	19.52
OpenMP	2	4	131.8	19.84
OpenMP	2	8	238.47	20.78
OpenMP	4	4	316.82	19.52
OpenMP	4	8	269.29	21.272

Calculated average times and used memory of each test case

Program Version	Avg Time (s)	StdDev Time (s)	Avg Memory (MB)	StdDev Memory (MB)
pthread	6.642857143	0.918072515	2.117714286	0.7391052506
MPI	1.558571429	0.5052533546	1.828571429	0.1561440117
OpenMP	149.54	126.3786038	20.13314286	0.6633117489

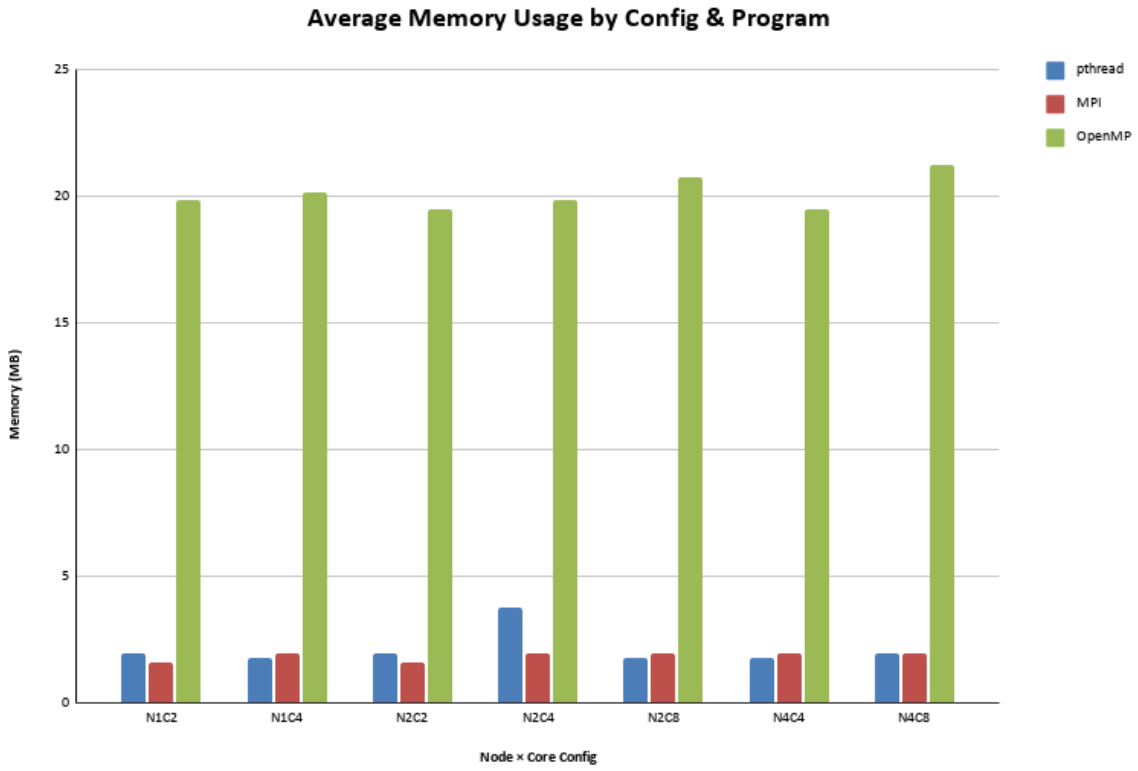
Calculated standard deviation for time and used memory

Performance Analysis Cont.



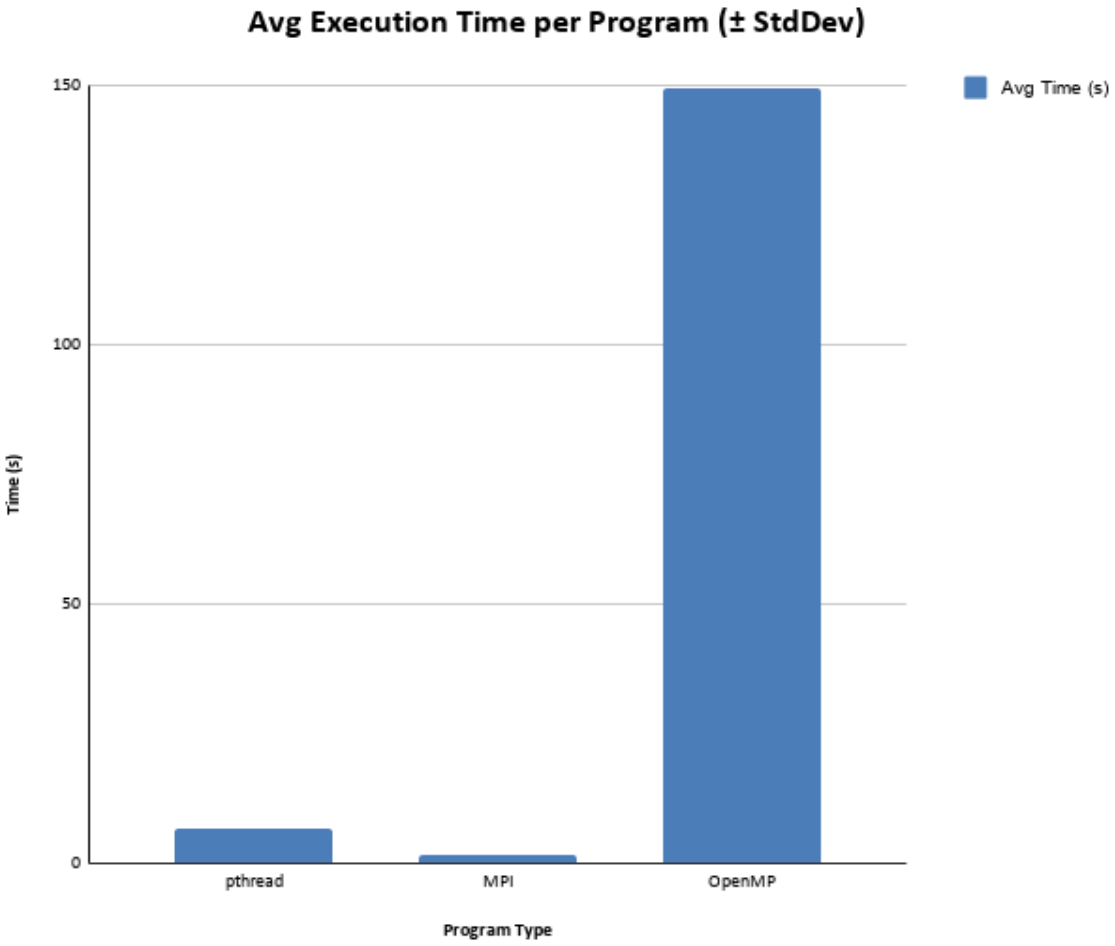
Average Execution time of each paradigm

Performance Analysis Cont.



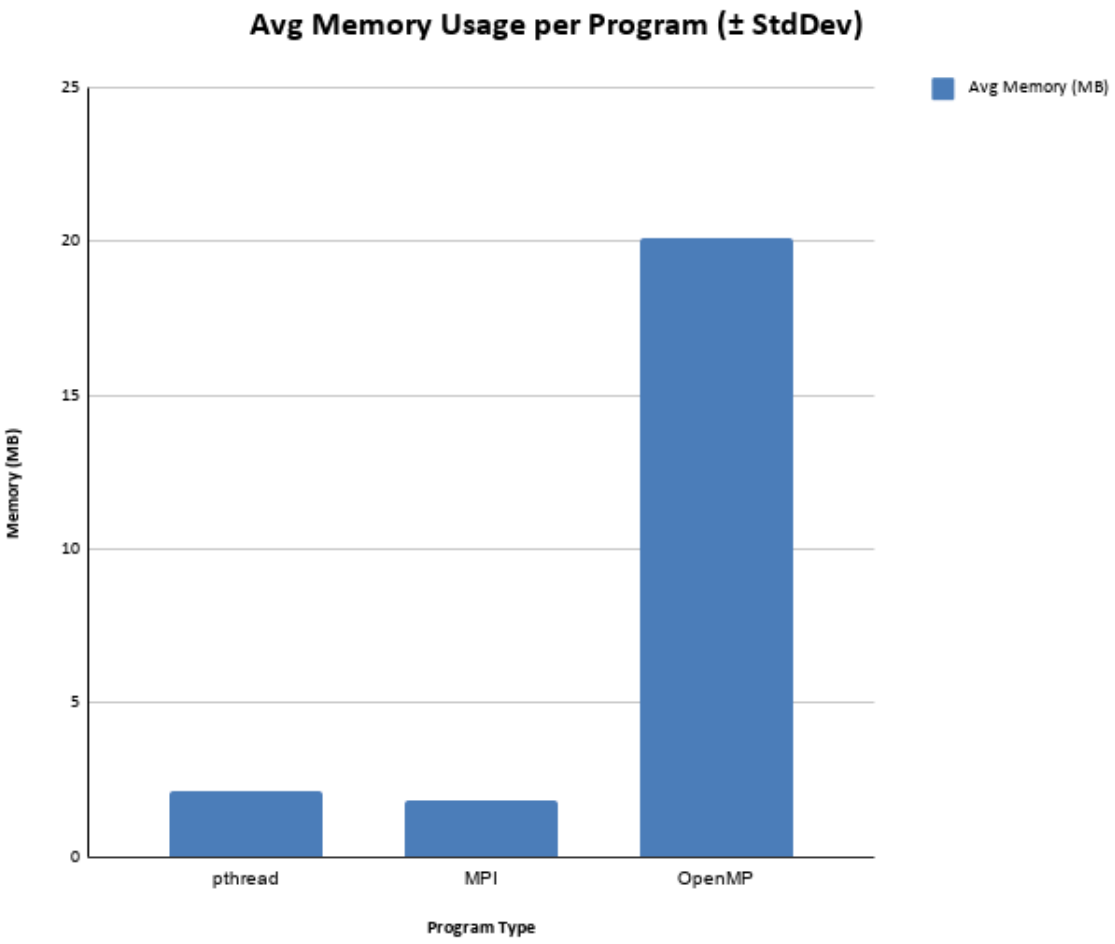
Average Memory Usage of each paradigm

Performance Analysis Cont.



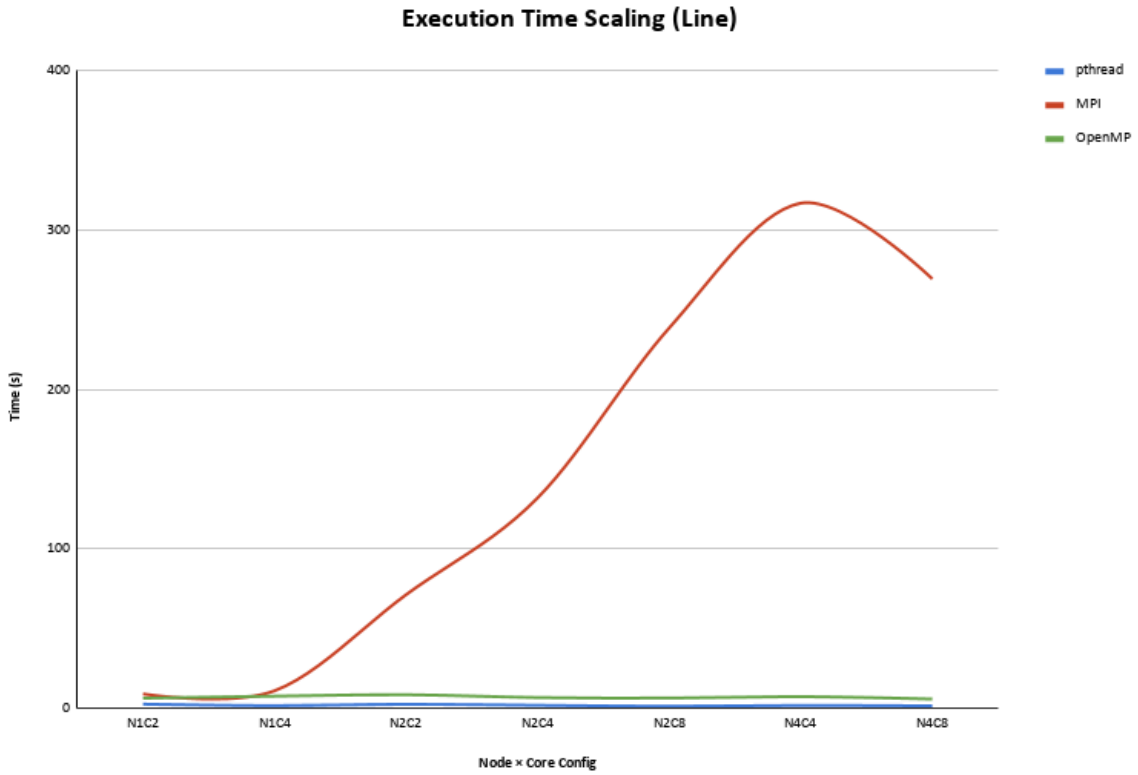
Standard deviation of execution time

Performance Analysis Cont.



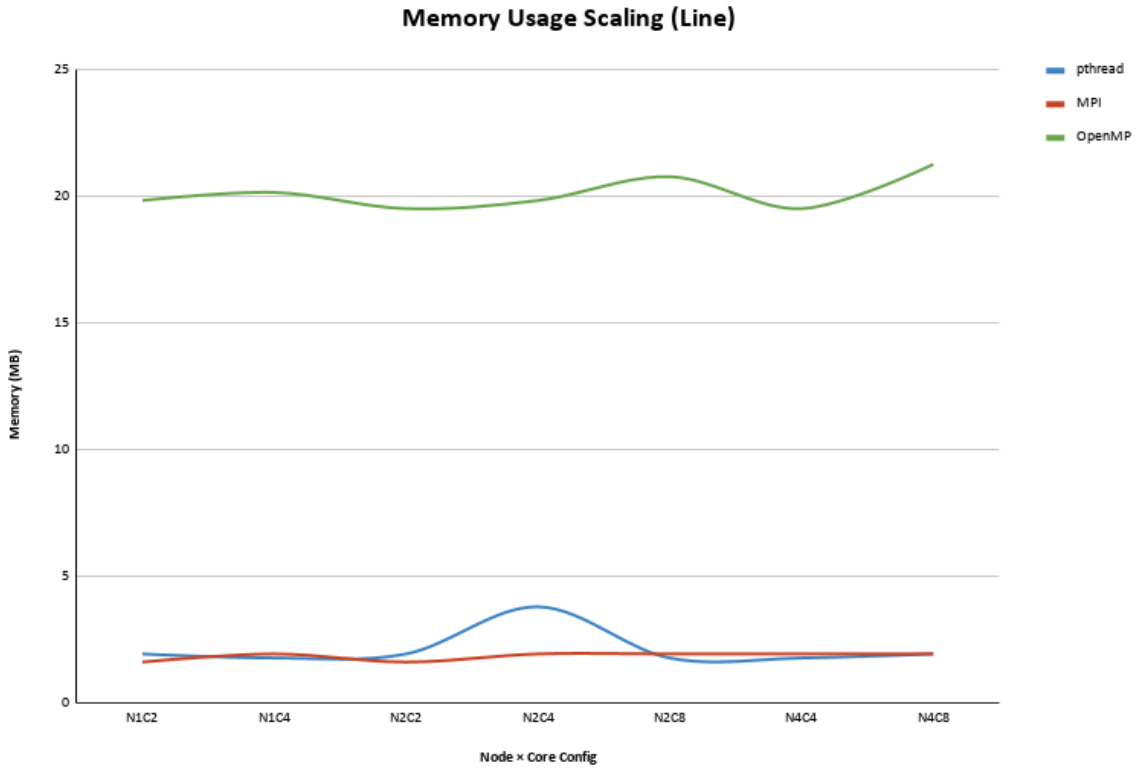
Standard deviaton of memory usage

Performance Analysis Cont.



Execution time scaling

Performance Analysis Cont.



Memory usage scaling

Appendix - pthreads Controlling Scripts

3way-pthread/run.sh

```
#!/bin/sh
# Usage: ./run.sh {NUM_THREADS}
make -s NUM_THREADS=$1 && ./3way-pthread-$1
```

3way-pthread/schedule.sh

```
#!/bin/sh
# Usage: ./schedule.sh {time} {mem-per-cpu} {num_cores}
sbatch --time=$1 --mem-per-cpu=$2 --cpus-per-task=$3 --ntasks=1 --nodes=1 --
constraint=moles ./run.sh $3
```

Appendix - OpenMP Controlling Scripts

3way-openmp/run.sh

```
#!/bin/sh
# Usage: ./run.sh {NUM_THREADS}
make -s NUM_THREADS=$1 && ./3way-openmp-$1
```

3way-openmp/schedule.sh

```
#!/bin/sh
# Usage: ./schedule.sh {time} {mem-per-cpu} {num_cores}
sbatch --time=$1 --mem-per-cpu=$2 --cpus-per-task=$3 --ntasks=1 --nodes=1 --
constraint=moles ./run.sh $3
```

Appendix - MPI Controlling Scripts

Steps to do MPI testing:
Enter the following:

```
module load CMake/3.23.1-GCCcore-11.3.0 foss/2022a OpenMPI/4.1.4-GCC-11.3.0
CUDA/11.7.0
```

Then:

```
bash mpi/submit_tests.sh
```

After the tests have all run, combine the results to a results.csv file:

```
bash mpi/collect_results.sh
```

3way-mpi/submit_test.sh

```
#!/bin/bash
# submit_tests.sh
# Most of this is Claud Sonnet 4.6, I was trying to get a prototype working quickly
for this .sh file, Comments are from me learning what the code does.
# Submit MPI pt2 test jobs for all parameter combinations.
# Run from the project root: bash mpi/submit_tests.sh
#
# Usage:
#   bash mpi/submit_tests.sh            # submit all combinations
#   bash mpi/submit_tests.sh --dry-run  # print sbatch commands without submitting

set -uo pipefail

DRY_RUN=false
if [[ "${1:-}" == "--dry-run" ]]; then
  DRY_RUN=true
  echo "[dry-run] No jobs will be submitted."
fi

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

# Make sure binary is gtg
if [[ ! -f "$PROJECT_DIR/bin/pt2" ]]; then
  echo "Binary not found. Building..."
  make -C "$PROJECT_DIR" mpi
fi

# Create results dir to put all of our results into
mkdir -p "$PROJECT_DIR/results"

# ── Parameter space ──────────────────────────────────────────────────────────
CORES_LIST=(1 2 4 8 16 32)
NODES_LIST=(1 2 4 8)
# Memory values as understood by Slurm (M = MiB, G = GiB) -> This seems to work so
far to the best of my testing.
MEM_LIST=("64M" "128M" "512M" "1G" "1536M" "3G")
# ─────────────────────────────────────────────────────────────────────────────

# Skip jobs that exceed beocat mole nodes
MAX_CORES_PER_NODE=40
```

```
submitted=0
skipped=0

for nodes in "${NODES_LIST[@]}"; do
  for cores in "${CORES_LIST[@]}"; do

    # skip job if the max cores go over the limit
    if (( cores > MAX_CORES_PER_NODE )); then
      echo "[skip] nodes=$nodes cores=$cores  (exceeds $MAX_CORES_PER_NODE cores/
node)"
      (( skipped++ )) || true
      continue
    fi

    for mem in "${MEM_LIST[@]}"; do

      JOB_NAME="mpi_n${nodes}_c${cores}_m${mem}"

      #creates a passable arg list to Slurm to get it to work
      SBATCH_ARGS=(
        --job-name="$JOB_NAME"
        --nodes="$nodes"
        --ntasks-per-node="$cores"
        --mem-per-cpu="$mem"
        --constraint=moles
        --time=02:00:00
        --output="$PROJECT_DIR/results/${JOB_NAME}_%j.out" #output file
        --error="$PROJECT_DIR/results/${JOB_NAME}_%j.err" #error output file
        "$SCRIPT_DIR/slurm_job.sh"
        "$nodes" "$cores" "$mem"
      )

      if $DRY_RUN; then #just for testing, if its a dry run then just echo what
should have been passed as if it were actually passing in information
        echo "sbatch ${SBATCH_ARGS[*]}"
      else
        if sbatch "${SBATCH_ARGS[@]}"; then
          echo "Submitted: nodes=$nodes cores=$cores mem=$mem"
          (( submitted++ )) || true
        else
          echo "[FAILED]  nodes=$nodes cores=$cores mem=$mem"
          (( skipped++ )) || true
        fi
      fi

    done
  done
done

if ! $DRY_RUN; then
  echo ""
  echo "Total submitted: $submitted  |  Skipped: $skipped"
  echo "Results will appear in: $PROJECT_DIR/results/"
  echo "Monitor with: squeue -u \$USER"
fi
```

3way-mpi/slurm_job.sh

```
#!/bin/bash
# Written by Claud Sonnet 4.6 for now so that I didn't have to wait on a teamate
before I started testing.
#SBATCH --job-name=mpi_pt2
#SBATCH --constraint=moles
#SBATCH --time=02:00:00
#SBATCH --output=results/mpi_%j.out
#SBATCH --error=results/mpi_%j.err

# Arguments passed from submit_tests.sh
NODES=$1
CORES=$2       # cores per node (= ntasks-per-node)
MEM=$3         # memory per core, e.g. "512M" or "1G"
VERSION="MPI pt2"

TOTAL_TASKS=$(( NODES * CORES ))

# Capture max RSS via /usr/bin/time -v; redirect program output to per-job file
OUTFILE="$SLURM_SUBMIT_DIR/results/output_mpi_${SLURM_JOB_ID}.txt"
TIMEFILE="$SLURM_SUBMIT_DIR/results/time_mpi_${SLURM_JOB_ID}.txt"

# Verify binary exists before running
if [[ ! -x "$SLURM_SUBMIT_DIR/bin/pt2" ]]; then
    echo "ERROR: binary not found or not executable: $SLURM_SUBMIT_DIR/bin/pt2" >&2
    exit 1
fi

# Use bash SECONDS as a reliable wall-clock fallback
SECONDS=0
/usr/bin/time -v mpirun -np "$TOTAL_TASKS" \
    "$SLURM_SUBMIT_DIR/bin/pt2" /homes/eyv/cis520/wiki_dump.txt > "$OUTFILE" 2>
"$TIMEFILE"

EXIT_CODE=$?
WALL_SECONDS=$SECONDS

# Parse elapsed wall-clock time and max RSS from /usr/bin/time -v output
ELAPSED=$(grep "Elapsed (wall clock)" "$TIMEFILE" | awk '{print $NF}')
MAX_RSS=$(grep "Maximum resident set size" "$TIMEFILE" | awk '{print $NF}')

# Fall back to SECONDS-based timer if /usr/bin/time -v parsing failed
if [[ -z "$ELAPSED" ]]; then
    printf -v ELAPSED "%d:%02d:%02d" \
        $((WALL_SECONDS / 3600)) \
        $(( (WALL_SECONDS % 3600) / 60 )) \
        $((WALL_SECONDS % 60))
fi
[[ -z "$MAX_RSS" ]] && MAX_RSS="N/A"

# Warn loudly if the job failed
if [[ "$EXIT_CODE" -ne 0 ]]; then
    echo "WARNING: mpirun exited with code $EXIT_CODE — timing may be invalid" >&2
fi

# Write this job's result to its own file; collect_results.sh will merge them all
```

```
later
echo "$VERSION,$NODES,$CORES,$TOTAL_TASKS,$MEM,$ELAPSED,$MAX_RSS,$EXIT_CODE" \
    > "$SLURM_SUBMIT_DIR/results/result_${SLURM_JOB_ID}.csv"
```

Appendix - Sample Output

(It wraps around into a second column)

```
0: 125
1: 125
2: 125
3: 125
4: 125
5: 125
6: 125
7: 125
8: 125
9: 124
10: 226
11: 195
12: 125
13: 226
14: 195
15: 125
16: 125
17: 125
18: 125
19: 125
20: 125
21: 226
22: 125
23: 125
24: 125
25: 195
26: 125
27: 125
28: 226
29: 125
30: 125
31: 125
32: 125
33: 125
34: 214
35: 226
36: 226
37: 226
38: 125
39: 125
40: 125
41: 125
42: 125
43: 125
44: 226
45: 226
46: 195
47: 217
48: 125
49: 226
50: 226
51: 125
52: 195
53: 125
54: 125
55: 125
56: 226
57: 125
58: 125
59: 125
60: 125
61: 125
62: 125
63: 125
64: 226
65: 125
66: 125
67: 125
68: 226
69: 194
70: 194
71: 226
72: 125
73: 226
74: 197
75: 125
76: 226
77: 125
78: 224
79: 226
80: 209
81: 125
82: 195
83: 226
84: 125
85: 125
86: 226
87: 226
88: 125
89: 226
90: 226
91: 125
92: 206
93: 226
94: 195
95: 195
96: 125
97: 226
98: 226
99: 195
```